

---

# **Plan de Pruebas — SIGAI-SES**

## **v1.0.0**

Securitas Colombia S.A.



## Plan de Pruebas — SIGAI-SES v1.0.0

---

### 1. Objetivo

[!IMPORTANT] Garantizar la **calidad, estabilidad y seguridad** del Sistema Integral de Gestión de Activos e Inventario (SIGAI-SES) mediante la ejecución de pruebas **unitarias, de integración, funcionales, de rendimiento y de seguridad**.

---

### 2. Alcance

[!NOTE] Este plan cubre **los 17 módulos** del sistema en su totalidad.

| Módulo                       | Cobertura                                           | Estado             |
|------------------------------|-----------------------------------------------------|--------------------|
| Autenticación y Autorización | Login, JWT, roles, permisos, cierre de sesión       | Cobertura completa |
| Gestión de Usuarios          | CRUD, asignación de roles, activación/desactivación | Cobertura completa |
| Catálogo de Items            | CRUD, validación de referencias únicas, soft delete | Cobertura completa |
| Activos                      | CRUD, validación de serial único, cambios de estado | Cobertura completa |
| Inventario Bulk              | Control de stock, alertas de mínimo, movimientos    | Cobertura completa |
| Clientes                     | CRUD, validación de NIT, soft delete                | Cobertura completa |
| Proyectos                    | CRUD, asociación con clientes, cambios de estado    | Cobertura completa |
| Proveedores                  | CRUD, categorías, soft delete                       | Cobertura completa |

| Módulo               | Cobertura                                               | Estado             |
|----------------------|---------------------------------------------------------|--------------------|
| Garantías            | Flujo completo de estados, cálculo de alertas           | Cobertura completa |
| Actas de Entrega     | Creación con detalles, generación de PDF, estados       | Cobertura completa |
| Desmontes y Triage   | Registro masivo, calificación técnica, cambio de estado | Cobertura completa |
| Movimientos (Kardex) | Registro automático, integridad transaccional           | Cobertura completa |
| Alertas              | Reglas, generación automática, cambios de estado        | Cobertura completa |
| Auditoría            | Registro de todas las acciones CRUD                     | Cobertura completa |
| Importación Excel    | Carga masiva con upsert, detección de tipo, validación  | Cobertura completa |
| Reportes             | Exportación Excel y PDF, streaming de grandes volúmenes | Cobertura completa |
| Búsqueda Global      | Resultados por tipo de entidad, mínimo 2 caracteres     | Cobertura completa |

---

---

### 3. Tipos de Pruebas

#### 3.1 Pruebas Unitarias

| Icono | Detalle                   | Valor                        |
|-------|---------------------------|------------------------------|
|       | <b>Herramienta</b>        | pytest + pytest-asyncio      |
|       | <b>Cobertura objetivo</b> | ≥ 80%                        |
|       | <b>Ejecución</b>          | Automática en cada push / PR |

---

| Componente                 | Casos a probar                                                    |
|----------------------------|-------------------------------------------------------------------|
| core/security.py           | Creación y validación de JWT, hash de contraseñas, refresh tokens |
| crud/*.py                  | Operaciones CRUD básicas, validaciones de unicidad, soft delete   |
| services/import_service.py | Parseo de Excel, upsert, normalización, detección de tipo         |
| api/deps.py                | Validación de tokens, obtención de usuario actual                 |
| schemas/*.py               | Validación de campos obligatorios, formatos, enumeraciones        |

### 3.2 Pruebas de Integración

| Icono | Detalle              | Valor                                 |
|-------|----------------------|---------------------------------------|
|       | <b>Herramienta</b>   | pytest + httpx (TestClient asíncrono) |
|       | <b>Base de datos</b> | MySQL de pruebas o SQLite en memoria  |

| Escenario                       | Descripción                                                     |
|---------------------------------|-----------------------------------------------------------------|
| Flujo completo de autenticación | Login → Obtener perfil → Refresh token → Logout                 |
| CRUD de inventario              | Crear item → Crear activo → Consultar → Actualizar → Eliminar   |
| Importación Excel               | Cargar archivo → Verificar upsert → Verificar resumen           |
| Flujo de garantía               | Crear caso → Enviar a proveedor → Recibir → Resolver → Entregar |
| Generación de acta              | Crear acta con detalles → Generar PDF → Descargar               |

| Escenario         | Descripción                                                              |
|-------------------|--------------------------------------------------------------------------|
| Alerta automática | Configurar regla → Reducir stock bajo mínimo → Verificar alerta generada |

### 3.3 Pruebas Funcionales (E2E)

| Icono | Detalle            | Valor                                                                        |
|-------|--------------------|------------------------------------------------------------------------------|
|       | <b>Herramienta</b> | Manual + scripts de automatización (Selenium/Playwright — <i>pendiente</i> ) |

| Módulo           | Casos de Prueba                                                   |
|------------------|-------------------------------------------------------------------|
| Login            | Credenciales válidas, inválidas, usuario inactivo, token expirado |
| Dashboard        | Visualización de métricas, búsqueda global                        |
| Inventario       | Filtros, paginación, creación, edición, eliminación, importación  |
| Garantías        | Creación, cambio de estados, alerta de estancamiento              |
| Actas de Entrega | Creación con detalles, generación PDF, cambios de estado          |
| Desmontes        | Registro masivo, triaje, cambio de estado                         |
| Alertas          | Visualización, filtros, cambio de estado, resolución              |
| Usuarios         | CRUD, cambio de rol, activación/desactivación                     |
| Reportes         | Exportación Excel, exportación PDF, grandes volúmenes             |

### 3.4 Pruebas de Rendimiento

| Icono | Detalle            | Valor                                    |
|-------|--------------------|------------------------------------------|
|       | <b>Herramienta</b> | Locust (locustfile.py existente)         |
|       | <b>Métricas</b>    | Latencia, throughput, tasa de error, CPU |

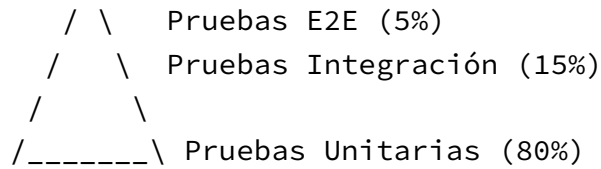
| Escenario                                        | Métrica Objetivo                  |
|--------------------------------------------------|-----------------------------------|
| 100 usuarios concurrentes consultando inventario | Respuesta < <b>500ms</b>          |
| 50 usuarios concurrentes importando Excel        | Procesamiento < <b>30s</b>        |
| Exportación de 50,000 registros a Excel          | Generación < <b>10s</b>           |
| Pico de 200 peticiones simultáneas               | Sin errores 5xx, CPU < <b>80%</b> |

### 3.5 Pruebas de Seguridad

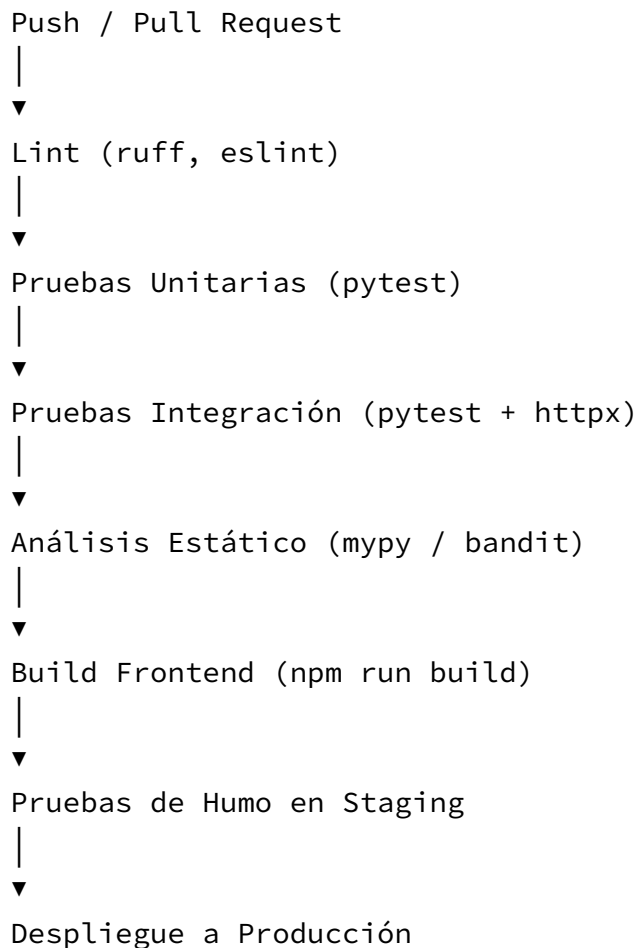
| Prueba               | Descripción                                                 |
|----------------------|-------------------------------------------------------------|
| Inyección SQL        | Validar que parámetros están sanitizados por SQLAlchemy     |
| XSS                  | Validar que las respuestas JSON no ejecutan scripts         |
| JWT tampering        | Verificar que tokens modificados son rechazados             |
| Acceso no autorizado | Verificar que endpoints protegidos requieren token válido   |
| Fuerza bruta         | Verificar rate limiting ( <i>pendiente de implementar</i> ) |
| OWASP Top 10         | Evaluación de cumplimiento (programada trimestralmente)     |

## 4. Estrategia de Pruebas

### 4.1 Pirámide de Pruebas



### 4.2 Flujo de Pruebas en CI/CD (GitHub Actions)



[!TIP] Este flujo garantiza que **cada cambio** pase por todas las capas de validación antes de llegar a producción.

## 5. Criterios de Aceptación

### 5.1 Criterios de Entrada

- ☑ Código compilado sin errores
- ☑ Pruebas unitarias ejecutadas localmente
- ☑ Code review aprobado (Pull Request)
- ☑ Documentación actualizada

### 5.2 Criterios de Salida

- ☑ Cobertura de pruebas  $\geq 80\%$
- ☑ **0** errores críticos o de alta prioridad
- ☑ Tiempo de respuesta **< 200ms** en operaciones críticas
- ☑ Tasa de fallos **< 0.5%**
- ☑ Pruebas de seguridad sin hallazgos críticos

[!WARNING] Si algún criterio de salida no se cumple, **no se autoriza el despliegue** hasta su corrección.

## 6. Herramientas

| Herramienta    | Propósito                                | Versión |
|----------------|------------------------------------------|---------|
| pytest         | Framework de pruebas unitarias           | 9.0+    |
| pytest-asyncio | Soporte para código asíncrono            | Última  |
| pytest-cov     | Medición de cobertura                    | Última  |
| httpx          | Cliente HTTP para pruebas de integración | Última  |
| Locust         | Pruebas de rendimiento                   | Última  |
| GitHub Actions | Integración continua                     | —       |



| Herramienta | Propósito                      | Versión |
|-------------|--------------------------------|---------|
| ruff        | Linter de Python               | Última  |
| ESLint      | Linter de TypeScript/React     | Última  |
| bandit      | Análisis de seguridad estático | Última  |
| coverage.py | Reporte de cobertura           | Última  |

---

## 7. Cronograma

| Fase                   | Duración           | Actividades                                   | Estado      |
|------------------------|--------------------|-----------------------------------------------|-------------|
| Pruebas Unitarias v1   | <b>Semana 1–2</b>  | Escribir tests para modelos, CRUD y schemas   | Completado  |
| Pruebas Integración v1 | <b>Semana 3–4</b>  | Probar flujos completos de API                | Completado  |
| Pruebas Funcionales    | <b>Semana 5–6</b>  | Validar módulos desde interfaz de usuario     | En progreso |
| Pruebas Rendimiento    | <b>Semana 7</b>    | Ejecutar Locust, optimizar cuellos de botella | Pendiente   |
| Pruebas Seguridad      | <b>Semana 8</b>    | Escaneo OWASP, pruebas de penetración         | Pendiente   |
| Ajustes y Cierre       | <b>Semana 9–10</b> | Corrección de defectos, reporte final         | Pendiente   |

Ver detalle completo del cronograma

## Plan de Pruebas — SIGAI-SES v1.0.0

| Semana | Lunes             | Martes          | Miércoles       | Jueves         | Viernes      |
|--------|-------------------|-----------------|-----------------|----------------|--------------|
| 1      | Setup pytest      | Tests models    | Tests CRUD      | Tests schemas  | Revisión     |
| 2      | Tests servicios   | Tests seguridad | Cobertura       | Refactor       | Reporte      |
| 3      | Setup integración | Test auth flow  | Test inventario | Test garantías | Test actas   |
| 4      | Test importación  | Test alertas    | Fixes           | Repetir fallos | Reporte      |
| 5-6    | E2E login         | E2E inventario  | E2E garantías   | E2E actas      | E2E usuarios |
| 7      | Setup Locust      | Escenario 1     | Escenario 2     | Escenario 3    | Optimización |
| 8      | OWASP scan        | SQL/XSS         | JWT/Acceso      | Rate limit     | Reporte      |
| 9-10   | Correcciones      | Validación      | Documentación   | Reporte final  | Cierre       |

[!NOTE] **Documento actualizado al:** Julio 2026 · **v1.0** · Responsable: Equipo de Calidad SIGAI-SES

Plan de Pruebas — SIGAI-SES · Securitas Colombia S.A. · Unidad de Seguridad Electrónica (SES)